

The Dot-Dot-Slash That Frameworks Hand You: CSPT Across Every Major Frontend Framework (Labs)

The Dot-Dot-Slash That Frameworks Hand You: CSPT Across Every Major Frontend Framework (Labs) - Author not stated, GitHub.

- Published: date not stated
- Original: <https://github.com/xssdoctor/cspt_research>
- Preserved from: https://github.com/xssdoctor/cspt_research (git) on 2026-09-09
- Repository commit: 2f195c4f6f2b497cd105da8005942fe4a02372b8
- Licence: see the repository

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

This reference is a source-code repository. The archive preserves its documentation at an exact commit; the code itself stays in a private mirror and is never checked out, built or run.

- Repository: <https://github.com/xssdoctor/cspt_research>
- Commit: [2f195c4f6f2b497cd105da8005942fe4a02372b8](https://github.com/xssdoctor/cspt_research/commit/2f195c4f6f2b497cd105da8005942fe4a02372b8)
- Documents preserved: 1

README.md

Blob [5b658307a253](#), 7378 bytes, at commit [2f195c4f6f2b](#).

Client-Side Path Traversal: A Cross-Framework Analysis

Comprehensive research into Client-Side Path Traversal (CSPT) vulnerabilities across 10 modern frontend frameworks, with runnable proof-of-concept labs and detection tooling.

Presented at: Critical Thinking Bug Bounty Podcast (March 19, 2026)

The Core Insight

Every modern frontend framework decodes URL parameters before passing them to developer code. When decoded params flow into `fetch()` calls, the browser normalizes `../` sequences *before* the HTTP request leaves — creating path traversal that bypasses server-side protections entirely.

Three things that make CSPT universal:

- 1. **No JavaScript encoding function encodes dots.** `encodeURIComponent('..') === '..'` — traversal survives all encoding.
- 2. **`fetch()` normalizes `../` before sending.** `fetch('/api/users/../../admin')` sends `GET /admin`. The browser does this, not the framework.
- 3. **Server-side rendering turns CSPT into SSRF.** Next.js Server Components, Remix loaders, SvelteKit `+page.server.ts` — all execute `fetch()` on the server with internal credentials and network access.

Quick Reference Matrix

| Framework | Params decoded? | `%2f` / in params? | Double-encode? | Catch-all slashes? |
Route breaks on `%2f`? | |-----| :~: | :~: | :~: | :~: | :~: | | **React Router v7** | Yes | Yes | Yes (two-step) |
Yes (`* splat`) | Yes | | **Next.js App Router** | Client: re-encoded / Server: Yes | Client: No / Server:
Yes | Version-dependent | Yes (`[...slug]`) | Depends on route type | | **Remix** | Yes (inherits RR) |
Yes | Yes | Yes (`$.tsx`) | Yes | | **Vue Router v4** | Yes | Yes | Not documented | Yes
(`/:pathMatch(.*)*`) | Yes | | **Nuxt 3** | Yes | Yes (via Vue Router) | Version-dependent | Yes | Yes | |
Angular | Yes | Yes (in paramMap) | Double-encodes on re-nav | N/A | No (routing preserves) | |
SvelteKit | Yes | Historical double-decode | Yes (historical) | Yes (`[...rest]`) | Yes | | **Ember.js** |
Yes | Decoded breaks route | Not documented | Yes (`/*path`) | Yes | | **SolidStart** | **No** | **No** |
N/A | Yes (`[...path]`) | No | | **Astro SSR** | Yes (`decodeURI`) | **No** (preserves `/`) | Yes (CVE bypass) |
Yes (`[...slug]`) | Depends on adapter |

Repository Structure

Research (`*-research/`)

Each framework has a dedicated research directory containing:

File	Contents
01-route-definitions.md	Route syntax, minification survival, detection regexes
02-source-to-sink.md	CSPT sources and sinks with minified patterns
03-encoding-behavior.md	Decoding pipeline with source code analysis
04-encoding-matrix.md	Complete encoding test matrix
05-caido-patterns.md	Regex patterns for automated detection
06-appendix.md	Source references, lab structure, build analysis

Frameworks covered: React Router, Next.js, Vue Router, Nuxt, Angular, SvelteKit, Astro, Ember.js, SolidStart, Remix

Labs (*-cspt-lab/)

Each lab is a real, runnable application demonstrating CSPT in the target framework.

Lab	Stack	Key Patterns
react-cspt-lab	React 19, React Router 7, TanStack Query	useParams fetch, splat routes, dangerouslySetInnerHTML chains
nextjs-cspt-lab	Next.js 15.3, App Router	Encoding differential: page params (re-encoded) vs route handlers (decoded)
vue-router-cspt-lab	Vue 3, Vue Router 4, TanStack Vue Query	route.params fetch, catch-all arrays, encoding split
nuxt-cspt-lab	Nuxt 3.17, Vue 3	Island payloads, server routes, H3
angular-cspt-lab	Angular 21, @angular/router	paramMap extraction, encoding behaviors
sveltekit-cspt-lab	SvelteKit 2, Svelte 5	+page.server.ts loaders, encoding differential
astro-cspt-lab	Astro 5, @astrojs/node	decodeURI behavior, middleware bypass
ember-cspt-lab	Ember.js 6, Embroider/Vite	Wildcard routes, triple-curlies, adapters
solidstart-cspt-lab	SolidStart 1.1, Solid.js 1.9	createResource, server functions, innerHTML
remix-cspt-lab	React Router 7 (Framework Mode)	Loaders (decoded), actions, .data endpoint, SSRF

Tools

cspt-analyzer — Caido Plugin

Passive framework detection with client-side path extraction and CSPT sink analysis. Integrates with the [Caido](#) HTTP proxy to detect framework fingerprints, extract dynamic route definitions from minified bundles, and identify CSPT source-to-sink chains.

cspt-devtools-extension — Chrome DevTools Extension

Chrome DevTools panel ("doctorscan") for detecting CSPT patterns directly in the browser. Identifies frameworks, extracts routes, and flags CSPT sources and sinks with per-framework encoding context. No traffic interception required.

Reference Documents

- `cspt-framework-notes.md` — Master reference document (54 KB) covering all frameworks with encoding comparisons, CVEs, and detection patterns
 - `react-cspt-deep-dive.md` — Detailed empirical analysis of React Router v7 encoding behavior with comprehensive encoding matrix
-

Key Findings

Encoding Differentials

The novel contribution of this research is documenting how each framework's encoding pipeline creates (or prevents) CSPT:

- **React Router v7** double-decodes through `safelyDecodeURI()` + `safelyDecodeURIComponent()` — `%252f` becomes `/`
- **Next.js App Router** re-encodes page params but leaves route handler params decoded — same framework, different attack surface
- **SolidStart** doesn't decode params at all — defense-in-depth by default
- **Astro** uses `decodeURI()` instead of `decodeURIComponent()` — `%2F` preserved, but encoded letters decode (CVE-2025-64765)
- **Vue Router** has the cleanest exploit: `route.params` decoded, `route.path` encoded — consistent and reliable

CVEs Referenced

CVE	Framework	Description
CVE-2025-64765	Astro	Middleware bypass via URL-encoded letters (<code>/%61dmin</code>)
CVE-2025-59414	Nuxt 3	Stored CSPT via island payload revival
CVE-2025-67647	SvelteKit	SSRF via <code>decode_pathname()</code> vs <code>url.pathname</code> discrepancy
CVE-2025-29927	Next.js	Middleware skip via <code>x-middleware-subrequest</code> header
GHSA-whqg-ppgf-wp8c	Astro	Double-encoding bypass of initial fix

Server-Side Amplification

When CSPT occurs in SSR contexts, it becomes SSRF with internal credentials:

- **Remix** loaders execute `fetch()` with server-side auth headers and internal network access
 - **SvelteKit** `+page.server.ts` load functions run on the server
 - **Next.js** route handlers and Server Components fetch with full server privileges
 - **Nuxt** server routes via H3 have access to internal services
-

Running the Labs

Each lab is a standalone application. Install dependencies and start the dev server:

```
# Example: React lab
```

```
cd react-cspt-lab
```

```
bun install
```

```
bun run dev
```

```
# Example: Next.js lab
```

```
cd nextjs-cspt-lab
```

```
bun install
```

```
bun run dev
```

Build to production to test detection against minified/bundled code:

```
bun run build
```

Author

xssdoctor — Offensive security researcher specializing in client-side vulnerabilities

License

This research is provided for educational and authorized security testing purposes only.

Archived from https://github.com/xssdoctor/cspt_research. Rendered offline from the local Markdown copy.